Project Proposal - ECE 379K

Team name: Hybrid Ray Tracer

Members: Mikail Sadic [MFS2447], Jett Vanbrocklin [JWV493], Arek Plate [AWP688], Po Han Ai [PA22435]

Date: September 21, 2026

1. What we are building

We are building a program that creates images of three-dimensional scenes by sharing work between a computer's main processor and graphics processor, then measuring how much faster this is than using the main processor alone.

Rendering requires checking which objects are visible, how they are lit, and which block the light. Repeating these calculations across a crowded scene can be slow. Faster rendering lets users inspect changes sooner; our study will identify when preparation and moving information between processors limit that improvement.

By December, users will be able to select a scene, set its camera, and save an image with solid-colored objects, lighting, and shadows. Deliverables include the renderer, a scene generator, a few demonstration scenes, reproducible instructions, and a performance study. We will render at 1920 x 1080 resolution, using smaller images during development and testing.

On the largest agreed benchmark scene, we target **~10x faster completion using multiple main-processor cores than one**, then **~2x faster with the graphics processor than the fastest tested multicore version: a goal of ~20x overall**. We will test core counts up to Vista's 72 cores and pursue larger gains where practical. These are goals to evaluate, not measured predictions.

Comparisons use the same scene, rendering rules, scene-search approach, and image settings. Total time includes preparation, transfers, and rendering, using the median of five runs on the same node. We also report component times and results across scene sizes. Our quality target is no visible differences from the reference in side-by-side review at native resolution on the same display by at least two reviewers, supported by tests of known visibility, lighting, and shadow cases. Approximate modes, if explored, will have their quality and speed tradeoffs reported separately.

2. Who is doing what

Team member	Owns
Jett (Member 1)	Builds parallel CPU scene preparation and camera controls; measures preparation scaling; tracks milestones.
Mikail (Member 2)	Builds GPU visibility calculations; verifies TACC access and builds; writes batch scripts.
Arek (Member 3)	Builds GPU lighting and shadows; measures rendering time and its components.
Po (Member 4)	Builds the single- and multicore CPU reference, scene generator, image comparisons, and automated CPU checks.

What we will do together: Agree on shared inputs and benchmark methods; integrate, review changes, and prepare the report and presentation. Everyone records contributions to planning, coordination, performance, and testing. Use GitHub issues, reviewed feature branches, and CPU checks on pull requests; run GPU comparisons on TACC before merging rendering changes.

Dependencies, and work meanwhile: Agree on shared inputs by Sept. 29. Member 2 starts with a fixed scene while Member 1 develops preparation. Member 3 tests lighting on known examples while awaiting visibility results. Member 4 starts the reference and generator; Members 1 and 3 help validate the reference before benchmarking.

2. Who is doing what (continued)

Milestones

By when	What exists	Who
Sept. 29: check-in 1	Shared inputs agreed; CPU/GPU access and test code verified on Vista	All; Member 2 verifies Vista
Oct. 26-30: check-in 2	CPU reference and first CPU/GPU image; benchmark scenes/settings agreed; initial checks and timings	All
Nov. 16-20: check-in 3	Working renderer with parallel preparation, lighting, shadows, multicore comparison, and preliminary results; scope frozen	All
Thanksgiving break; Nov. 30	Final tuning, scene runs, quality review, measurements, and presentation ready	Members 1, 3 measure; Members 2, 4 validate; all prepare slides
Dec. 1/3; Dec. 6	Presentation on assigned date; report and reproducible release Dec. 6	All

3. What could go wrong

What could go wrong	How we would notice early	What we would do instead
Vista access or build environment blocks testing	Test CPU/GPU access and code on Vista by Sept. 29	Seek Lonestar6 GPU access; continue CPU tests during delays.
Preparation, transfers, or GPU execution limit speedup	Initial timings in October; CPU scaling and component measurements by Nov. 20	Tune the largest cost; report achieved speedups and explain missed targets without weakening the reference.
Rendering errors or approximations pass casual inspection	From October integration, run known-case tests and image comparisons; inspect edges and shadows	Fix errors or disable approximations; retain the verified rendering mode.
Coursework, absences, or queue waits overwhelm Thanksgiving catch-up	Weekly review; require a working renderer and initial measurements by Nov. 20	Adopt reduced scope by Nov. 20; retain preliminary results if later runs are blocked.

Reduced scope: Render one fixed scene of spheres and a floor, with one light and shadows at 640 x 480. Keep parallel CPU preparation, GPU rendering, quality checks, and single- and multicore CPU comparisons. Drop camera controls and larger scenes; report measured performance even if targets are missed.

4. What we will need

What	Status	Needed by
Primary: TACC Vista Grace-Hopper (GH200); alternative: Lonestar6 A100 node. One node/job, one GPU plus CPU cores; estimated 50 node-hours total	Have allocation; CPU tested, GPU unverified. **Need from instructor** only if access tests fail; flag by Sept. 29.	Test by Sept. 29
C++/CUDA, OpenMP, CMake, CPU/GPU profiling tools	Can get/use TACC tools; verify compatibility. No paid licenses requested.	Sept. 29
GitHub repository; Classroom when available	Can create our own shared repository and migrate if required	Own repo Sept. 29; Classroom when provided
Repeatable generated scenes of increasing size, provisionally 10,000 to 1,000,000 simple objects; estimated scene data under 1 GB	Can create; no external assets needed. Fix benchmark sizes at October check-in.	First October; remaining Nov. 20